

Java File Handling

Managing the File System with java.io

codehive

codehive

The File Class

The `File` class from the **java.io** package allows us to work with files. It can be used to gather info about files, create new ones, and manage paths.

```
import java.io.File;

File myObj = new File("filename.txt");
```

Essential Methods

Method	Description
<code>canRead()</code>	Checks if the file is readable.
<code>canWrite()</code>	Checks if the file is writable.
<code>exists()</code>	Checks if the file actually exists.
<code>getName()</code>	Returns the name of the file.
<code>length()</code>	Returns the size of the file in bytes.
<code>delete()</code>	Deletes the file from the system.

1. Creating & Writing

To create a file, we use `createNewFile()`. To add text to it, we typically use the `FileWriter` class.

```
import java.io.FileWriter;
import java.io.IOException;

try (FileWriter writer = new FileWriter("filename.txt")) {
    writer.write("Java file handling is powerful!");
    System.out.println("Successfully wrote.");
} catch (IOException e) {
    System.out.println("An error occurred.");
}
```

2. Reading a File

The `Scanner` class is a common choice for reading the content of text files line by line.

```
import java.util.Scanner;
import java.io.File;

try {
    File myObj = new File("filename.txt");
    Scanner myReader = new Scanner(myObj);
    while (myReader.hasNextLine()) {
        String data = myReader.nextLine();
        System.out.println(data);
    }
    myReader.close();
} catch (Exception e) { ... }
```

Note: Always handle `FileNotFoundException` when attempting to read files that might not exist.

Summary Checklist

- ✓ Use `java.io.File` for metadata (size, name, existence).
- ✓ Use `java.io.PrintWriter` to create or overwrite content.
- ✓ Use `java.util.Scanner` for easy line-by-line reading.
- ✓ Always wrap file operations in **Try-Catch** blocks.

Persistence and Storage with

codehive

Handling Data Safely.